From LLMs to Retrieval

A compact teaching document for a one-hour, no-code-writing demonstration

Source lecture	Main ideas represented in this document
1. LLM Methodology & Ecosystem	Next-token prediction, sampling, model ecosystem
2. LLM as an Instrument	Function calling, PDF ingestion, OCR, cleaning, chunking, structured output
3. Embedding Strategies	Embeddings, vector search, bi-encoder, cross-encoder, reranking, evaluation

This is a purpose-built, paraphrased teaching dataset. It is not a copy of the original lecture slides. Repeated headers and page numbers are intentional so that cleaning can be demonstrated.

1. How a Language Model Generates Text

Next-token prediction

A language model generates text one token at a time. Given the text already produced, it calculates a score for every possible next token, converts the scores to probabilities, selects or samples a token, appends it to the sequence, and repeats. A token may be a complete word, part of a word, punctuation, or whitespace.

Sampling controls

Temperature changes the sharpness of the probability distribution. A low temperature favors the most likely continuation and makes generation more predictable. A high temperature spreads probability across less likely alternatives and increases diversity. Top-k limits the candidate set by count, while top-p keeps the smallest set whose cumulative probability reaches a chosen threshold.

What the model does not do

The model does not retrieve a complete stored sentence before answering. It generates the sequence step by step. This means a fluent answer can still be unsupported by the required source. When factual grounding matters, the system should first retrieve relevant evidence and then generate from that evidence.

> **Key question: Does a more creative sampling configuration make the model more knowledgeable? No. It changes how tokens are selected, not what evidence is available.**

2. Function Calling and Structured Output

Function calling

Function calling allows a language model to select an external tool and produce arguments that follow a declared schema. For example, a search_course_materials tool may require a query string and an integer top_k. The model chooses the tool and fills the arguments, but application code executes the actual search.

Execution boundary

The distinction is important: the model proposes a tool call; the backend validates the arguments, applies permissions, calls databases or APIs, handles errors, and returns the result. This separation makes the system more controllable than asking the model to imitate an external action in free text.

Structured output

A production result should have a predictable structure such as JSON with answer, source, page, and confidence fields. Schema validation catches missing fields and invalid types before data reaches a database, dashboard, automated workflow, or another tool.

> **Example tool input: {"query": "When is OCR required?", "top_k": 5}**

3. PDF Ingestion, Cleaning, and OCR

Selectable text versus scanned pages

A digital PDF may contain selectable text that a parser can extract directly. A scanned PDF may contain only page images. When no selectable text exists, optical character recognition is required to convert the image into machine-readable text before search or summarization.

Cleaning

Extracted text often contains repeated headers, page numbers, broken line wraps, duplicated whitespace, and boilerplate. Cleaning should remove retrieval noise while preserving meaning. The original page number should usually be kept as metadata even when it is removed from the searchable text.

Provenance

Each extracted unit should retain source information such as file name, page, section, and extraction method. Provenance allows users to verify the answer and enables the system to display citations or open the original page.

OCR is necessary when the page is an image of text rather than selectable text. OCR should not be the default for every PDF because it is slower and can introduce recognition errors.

4. Chunking Strategies

Why chunk documents

Long documents are divided into smaller units so that retrieval can operate at the paragraph or idea level. If chunks are too small, they may lose the context required to answer a question. If chunks are too large, they mix several topics and add noise.

Fixed chunks and overlap

Fixed-size chunking is simple and deterministic, but it may split a sentence or concept at a boundary. Overlap repeats a small region from the previous chunk, helping preserve context when an answer spans a boundary. The cost is additional storage, embedding work, and duplicate retrieval results.

Semantic or structure-aware chunks

Structure-aware chunking splits at headings, paragraphs, or topic shifts. It often produces more coherent units, but the chunks are not uniform and the implementation is more complex. The best strategy depends on document structure and must be evaluated rather than assumed.

> **A useful starting point is a moderate chunk size with limited overlap, followed by retrieval evaluation on representative questions.**

5. Embeddings and Vector Stores

Embeddings

An embedding model maps a query or text chunk to a dense numerical vector. Semantically related texts should point in similar directions. Cosine similarity compares the angle between vectors and is commonly used to rank candidate chunks.

Semantic retrieval

A semantic query can retrieve a relevant chunk even when the wording differs. For example, a question about recognizing text from page images may retrieve a chunk about OCR even if the exact acronym does not appear in the query.

Vector stores

A vector store keeps the text, embedding, and metadata together and supports nearest-neighbor search. FAISS is primarily a fast vector-search library. Chroma adds database-style features such as collections, metadata filtering, and persistence. The same embedding model and preprocessing must be used for documents and queries.

> **Similarity is a ranking score, not a calibrated probability that a chunk is correct.**

6. Bi-Encoder, Cross-Encoder, and Reranking

Bi-encoder retrieval

A bi-encoder encodes the query and each document independently. Document vectors can be computed once and indexed. At query time, only the query vector is computed, so the method scales to large collections. The tradeoff is that relevance is estimated from two separate vectors without full token-level interaction.

Cross-encoder reranking

A cross-encoder receives the query and a candidate chunk together and directly predicts relevance. Tokens from the query and chunk can interact inside the Transformer, which often improves precision for negation, constraints, and subtle distinctions. The cost is much higher because the model must run once for every query-candidate pair.

When reranking helps

Reranking is valuable when only a few chunks fit into the final context, when precision is important, or when first-stage retrieval returns broadly related but incomplete passages. A common workflow retrieves 20 to 100 candidates with a bi-encoder and reranks them with a cross-encoder.

When reranking should be avoided

Reranking may be skipped when very low latency is the top priority, when the corpus is tiny and direct scoring is already cheap, when the initial retrieval is already sufficiently precise, or when the task is broad exploratory search rather than exact evidence selection.

> **If the correct chunk is missing from the first-stage candidate set, the cross-encoder cannot recover it. The first stage protects recall; reranking improves precision.**

7. Retrieval Evaluation and the Final Pipeline

Evaluate retrieval separately

A grounded answer requires the right evidence. Retrieval should therefore be evaluated before the generation step. Create benchmark queries with known relevant pages or chunks and compare chunking strategies, embedding models, top-k values, and reranking configurations.

Core metrics

Recall@k asks whether a relevant result appears in the first k positions. Precision@k measures how many of the first k results are relevant. Mean Reciprocal Rank rewards systems that place the first relevant result near the top. These metrics describe different priorities and should be selected according to the application.

End-to-end flow

A practical pipeline is: extract and clean the PDF, preserve metadata, create chunks, compute embeddings, store or index the vectors, embed the user query, retrieve top candidates, optionally rerank them, return structured evidence, and let the language model formulate an answer that cites the source page.

> **The quality of a retrieval-augmented system is often determined before the language model generates a single token.**